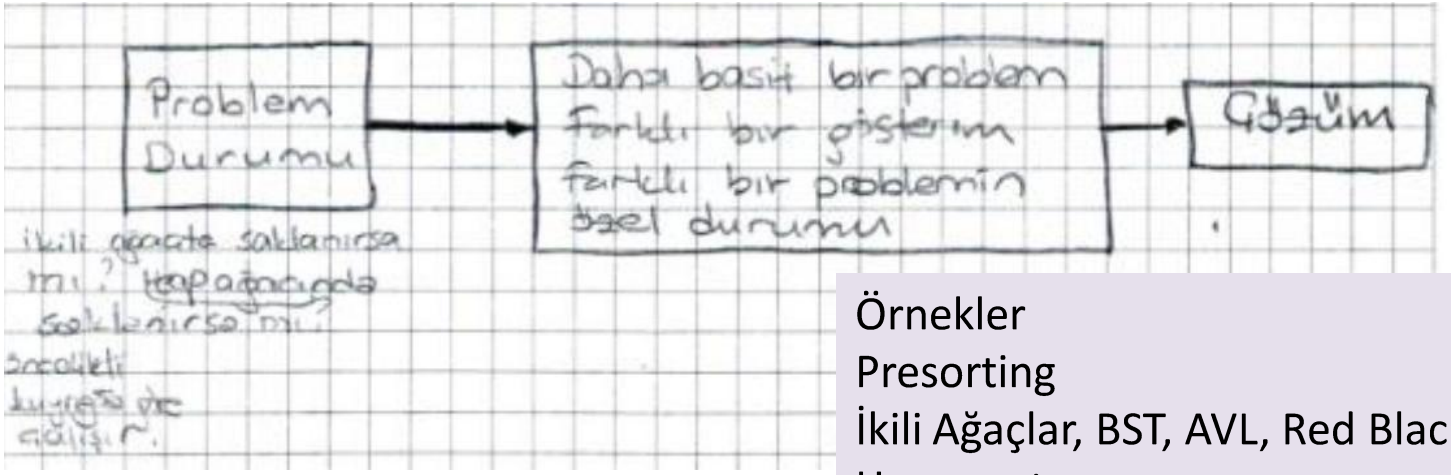
Dönüştür Yönet (Transform & Conquer)

- Bu teknikteki algoritmalar dönüştürme işlemini kullanarak problemi çözerler.Tasarımı daha kolay hale getirir.
- Aynı problemin daha basit/uygun ifade ediliş biçimidir.
- Aynı problemin farklı ifade ediliş biçimidir.
- Farklı ifade ediliş biçimine daha önce çözüm bulunduğu için dönüştürülür.



Örnekler
Presorting
İkili Ağaçlar, BST, AVL, Red Black,…
Heap sort
Gaussian elimination

Dönüştür Yönet (Transform & Conquer)

Örnekler

- Presorting
- İkili Ağaçlar, BST, AVL, Red Black,…
- Heap sort
- Gaussian elimination

Presorting (Önce Sırala)

- Presorting (Önce Sırala)
- Listeler dahil birçok problemin çözümüne başlanmadan önce elemanlar sıralanırsa problemi çözmek
- kolaylaşır.
- - Arama
- - Orta değerin bulunması
- - Tek olan elemanların kontrolü (element uniqueness)
- - Birçok geometrik algoritmada presorting kullanılır.
  - Noktaların koordinatlarına göre sıralanması

Presorting Kullanarak Arama

- Problem: K değerini A[0..n-1] dizisinde arama
- Presorting Temelli Arama Algoritması
- 1. Kısım: Etkili bir sıralama algoritması ile diziyi sırala
  - En kötü durumda sıralamada O(nlogn) garantidir. (Mesela MergeSort)
- 2. Kısım: İkili Aramayı Uygula O(logn)
- Performans= O(nlogn) + O(logn) = O(nlogn)

Presorting Kullanarak Teklik Kontrolü (aranan elemandan dizide elemandan bir tane mi var)

- 1. Kısım: Etkili bir sıralama algoritması ile diziyi sırala
- En kötü durumda sıralamada O(nlogn) garantidir. (Mesela MergeSort)
- 2. Kısım: Dizide eleman eleman gidip komşulara bak. O(n)
- Performans = O(nlogn)+O(n) = O(nlogn)
- Brute Force Kullanılsaydı: Sırasız dizide her elemanı diğer bütün elemanlar ile karşılaştır.
- Performans = $O(n^2)$
- Presort ile daha iyileştirilmiş oldu. Alternatif olarak Hashcode kullanılabilir. Bu durumda sadece aynı indexteki listeye bakılacaktır. En kötü durumda O(n)

Presorting Kullanarak Dizide En Sık Geçen Elemanın Bulunması

- Eğer presort kullanılmaz ise bütün elemanlar diğer bütün elemanlar ile karşılaştırılmalıdır. (Brute Force yaklaşım)

- A={1,2,1,1,2,3,4,5,6,9,8,7,3,2,3,1}

$$C(n)=(n-1)+(n-2)+(n-3)+.....+1=\sum_{i=1}^{n} i-1=\frac{(n-1)\,n}{2}=O(n^2)$$

Presort Kullanılınca

```
PreSortMode(A[0..n-1])
A dizisini sırala
i=0 , mf=0;  //en yüksek frekansa sahip elemanı tutar
while(i<n)
        rl=1, rv=A[i];
        while(i+rl < n && A[i+rl]==rv)
                rl=rl+1;
        endwhile
        if(rl>mf)
                mf=rl;
                mv=rv;
        endif
endwhile
return mv;
```

rl: O anki seçilen elemanın kaç adet olduğu
rv: O an seçilen elemanın ne olduğu

Sıralama için: HeapSort Kullan bu durumda: O(nlogn)
Yukarıdaki iki while kısmı için: O(n)
Toplam Maliyet = O(n)+O(nlogn) = O(nlogn)

İkili Arama Ağaçları (Binary Search Trees)

- İkili Arama ağaçlarında performans ağacın yüksekliğine bağlıdır. n elemanlı bir ağaçta performans: $\log n \leq h \leq n-1$ normal bir ikili arama ağacında
- En kötü durumda elemanın bulunması: O(n)
- Ortalama Durumda: O(logn)
- En iyi durumda O(1) kökte olma durumu

- En Kötü Durumu iyileştimenin iki alternatif yolu
- **-AVL Ağaçları**
- **- Kırmızı-Siyah Ağaçlar (Red-Black)**

- Yükseklik: Hiç düğümü olmayan bir ağacın yüksekliği tanımsız olacağı için -1 olarak kabul edilir. Tek düğümlü bir ağacın yüksekliği sıfırdır. Yani bir ağacın yüksekliği kökten en uzaktaki yaprağa olan kenarsayısıdır.

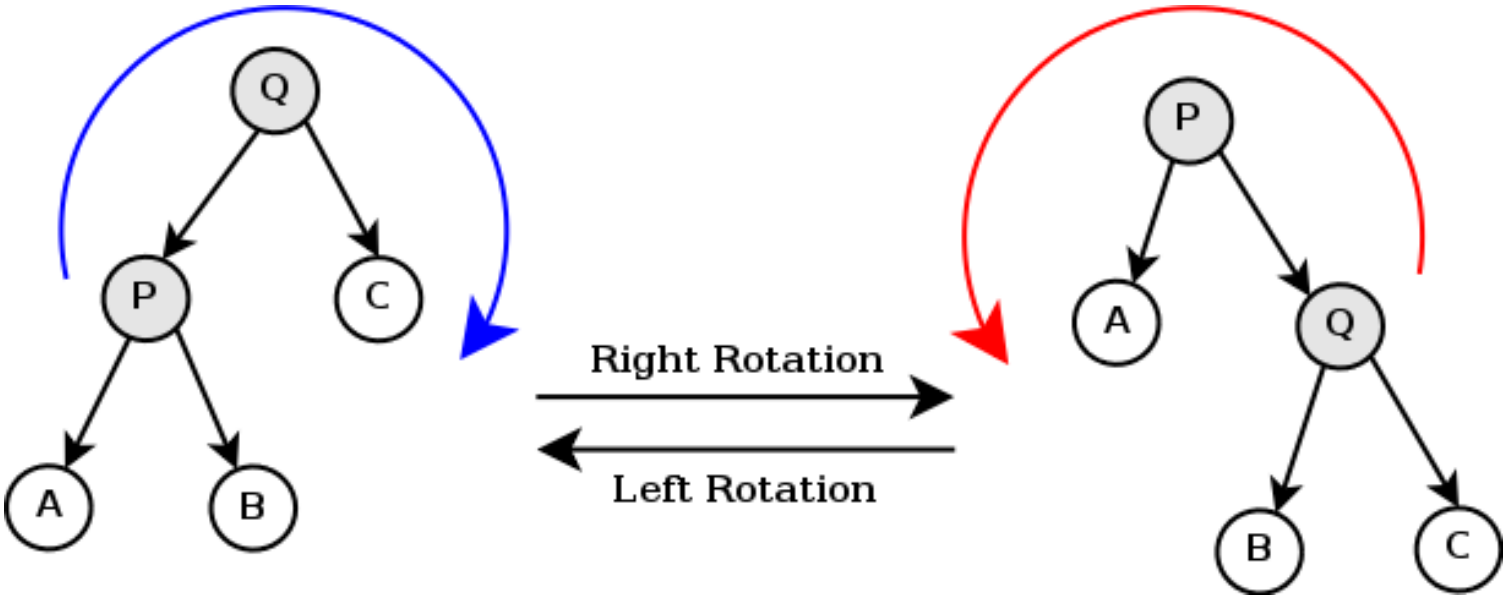
AVL Ağaçları

- Bir ikili arama ağacı eğer aşağıdaki koşulları sağlıyorsa dengeli (AVL) bir ağaçtır.
- - Yaprak ve kök olmayan her düğümüm mutlaka bir kardeşi olmalıdır.
- - Kardeşlerin yükseklikleri eşit veya en fazla 1 fark olmalıdır.

AVL Ağaçları

- Dengesiz bir ağaç hafif ya da ağır diye nitelendirilebilir. Bu nitelemeler aşağıdaki durumlara bağlıdır
- - Eğer bir düğümün kardeşinin yüksekliği daha fazla ise o düğüm hafiftir.
- - Hafif olmayan her düğüm ağırdır.
- - Bir düğüm sol-ağır şeklinde nitelendiriliyorsa düğümün sol çocuğunun yüksekliği sağ
- çocuğunun yüksekliğinden daha fazladır.
- - Aynı şekilde bir düğüm sağ-ağır şeklinde nitelendiriliyorsa düğümün sağ çocuğunun yüksekliği
- sol çocuğunun yüksekliğinden daha fazladır.

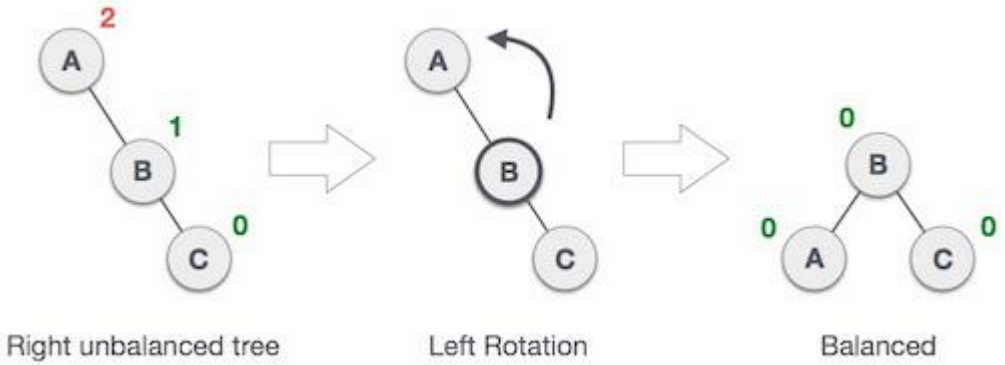
Ağaç döndürme (tree rotation)



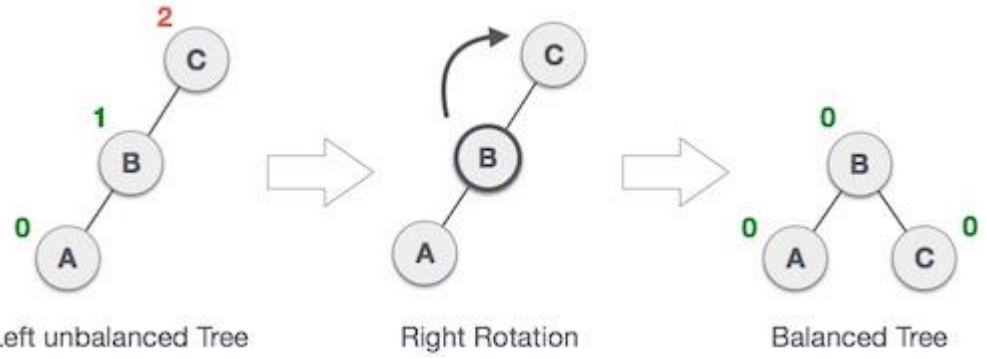
https://en.wikipedia.org/wiki/Tree_rotation

Ağaç döndürme (tree rotation)

Left Rotation

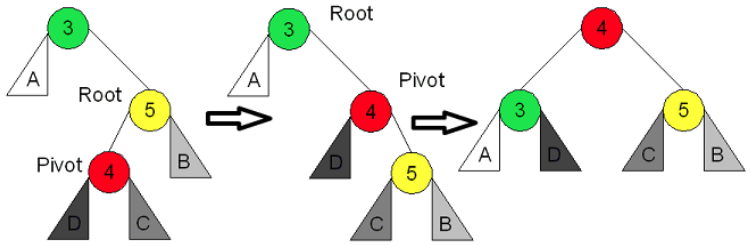


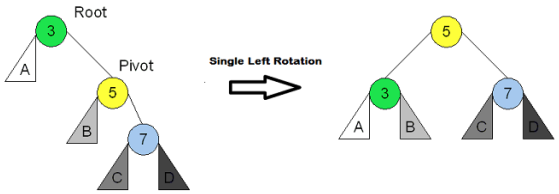
Right Rotation



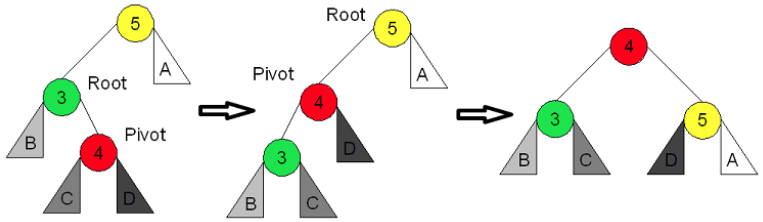
Döndürme işleminin belirlenmesi

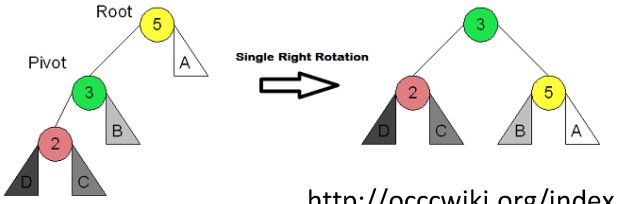
- If (Ağaç sağ ağır) (h(sag(d))-h(sol(d))==2)
  - If (sağ alt ağaç sol ağır ise)
    - Çift dönüş
  - Else
    - Tek dönüş





- Else if (Ağaç sol ağır) (h(sag(d))-h(sol(d))==-2)
  - If (sol alt ağaç sağ ağır)
    - Çift dönüş
  - Else
    - Tek dönüş

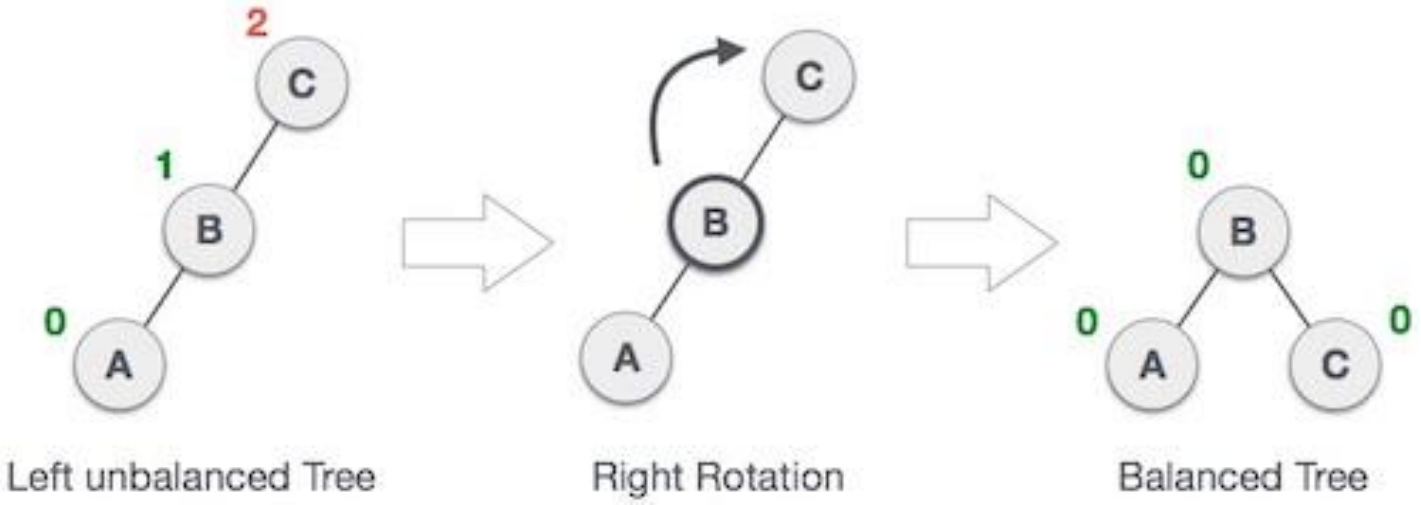




http://occcwiki.org/index.php/AVL_Trees

Örnek: Ağaç sol ağır

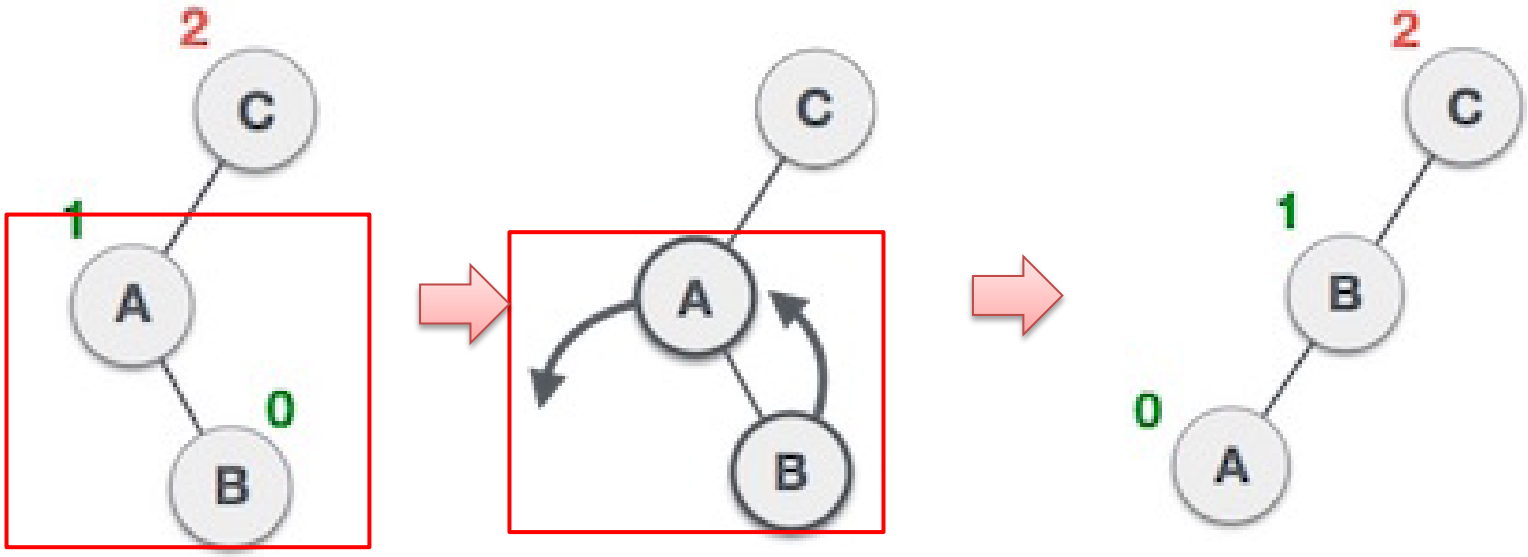
Sol alt ağaç sol ağır: Sağa tek dönüş durumu

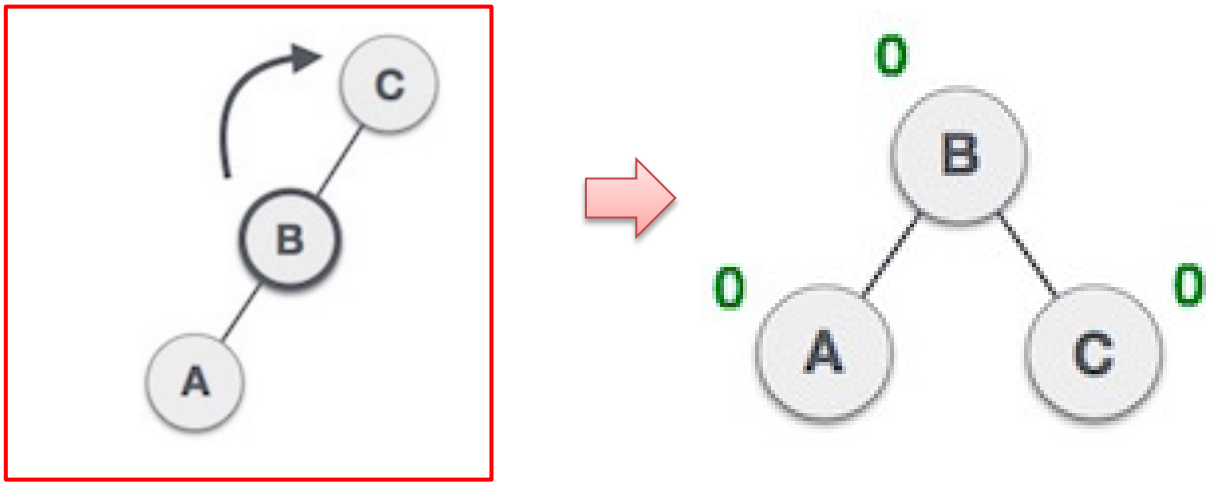


https://www.tutorialspoint.com/data_structures_algorithms/avl_tree_algorithm.htm

Örnek: Ağaç sol ağır

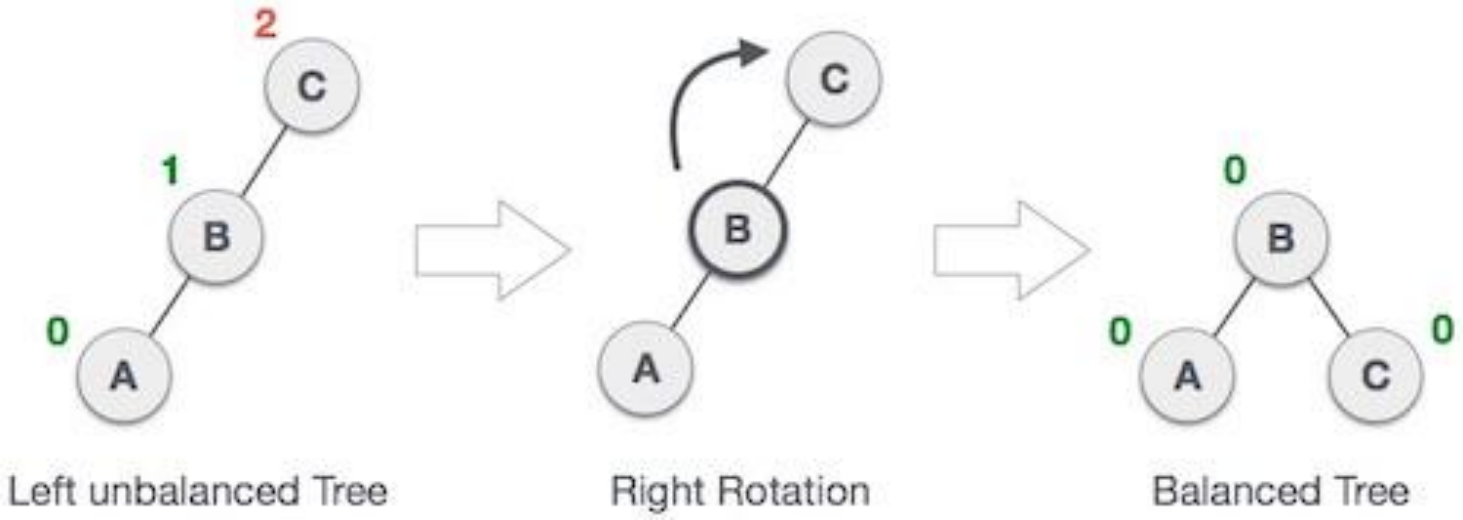
Sol alt ağaç sağ ağır: Sola –sağa dönüş (çift dönüş) durumu





Örnek: Ağaç sağ ağır

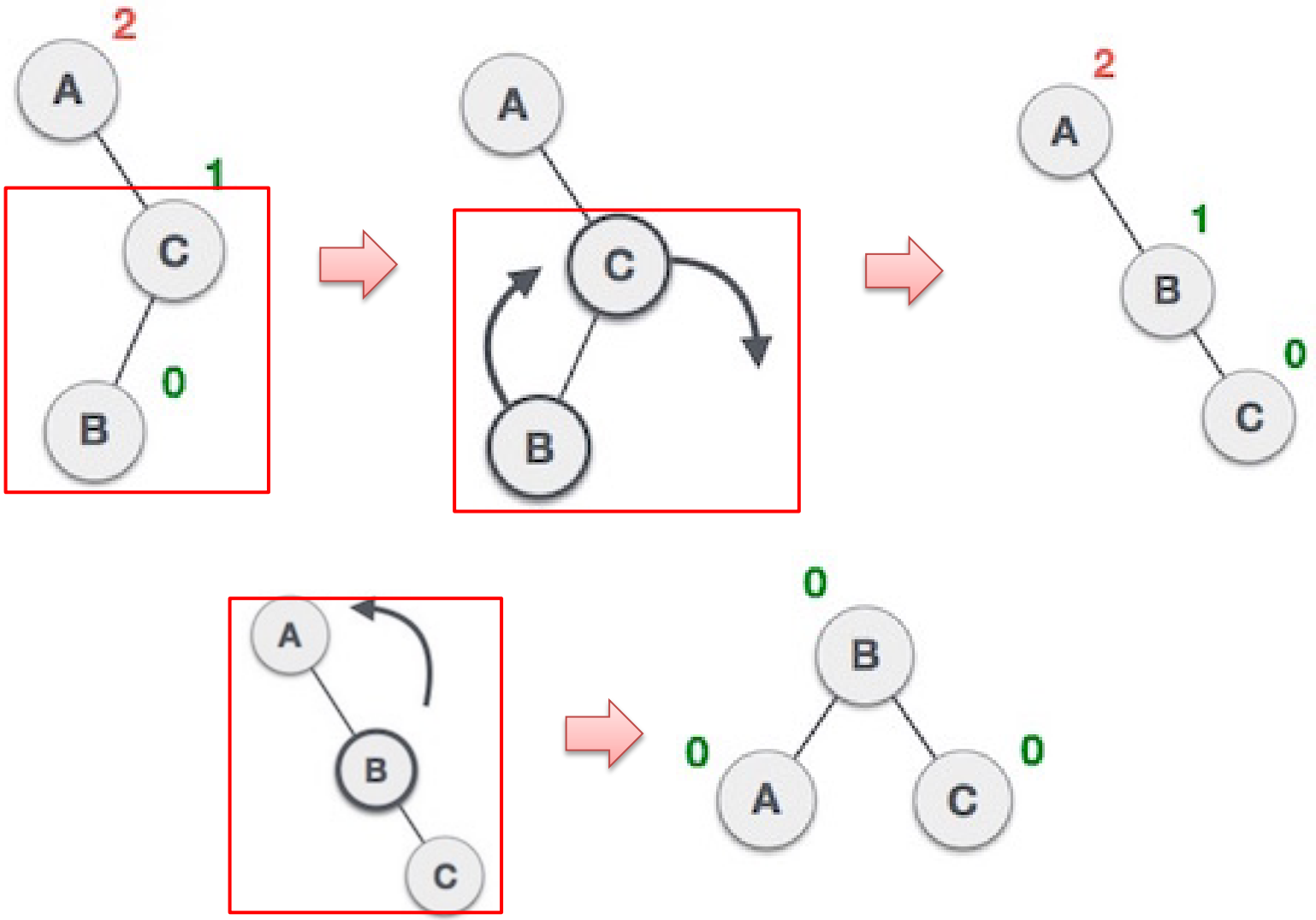
Sol alt ağaç sol ağır: Sağa tek dönüş durumu



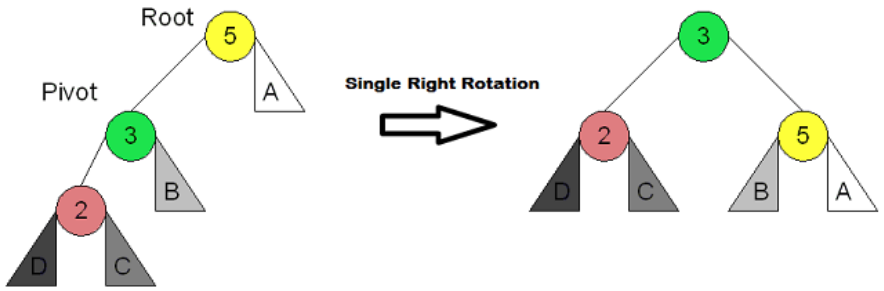
https://www.tutorialspoint.com/data_structures_algorithms/avl_tree_algorithm.htm

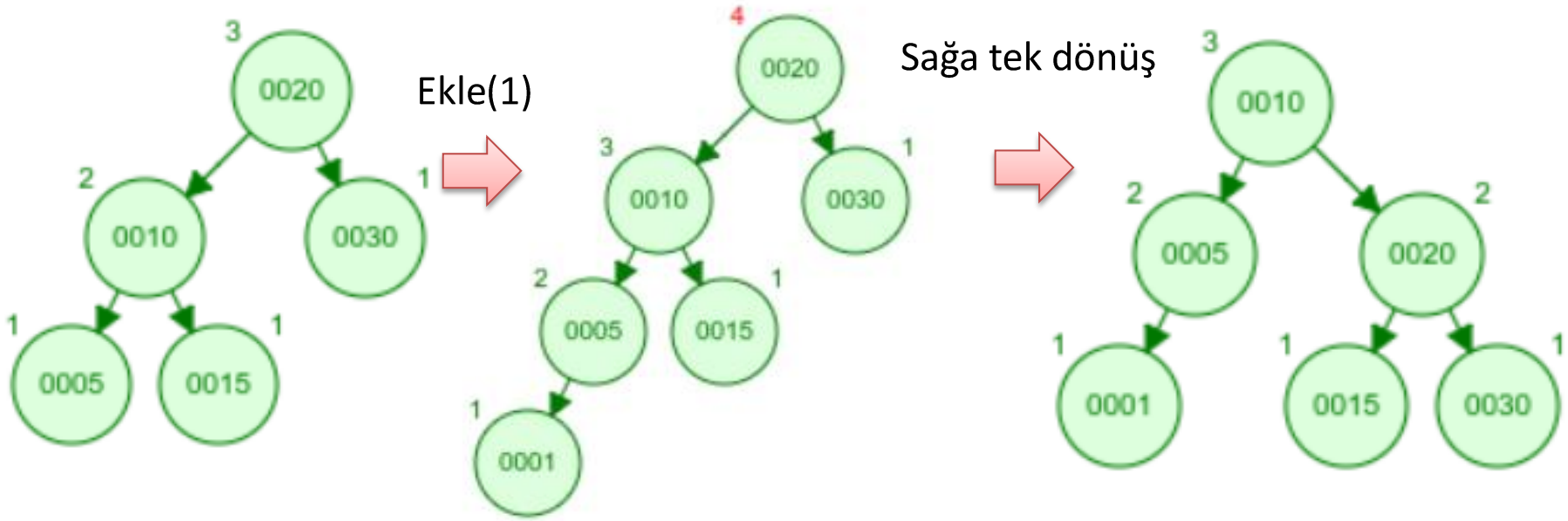
Örnek: Ağaç sağ ağır

Sağ alt ağaç sol ağır: Sağa –sola dönüş (çift dönüş) durumu



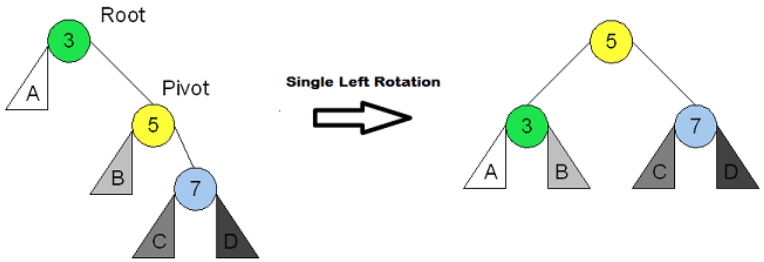
Örnek: sağa tek döndürme

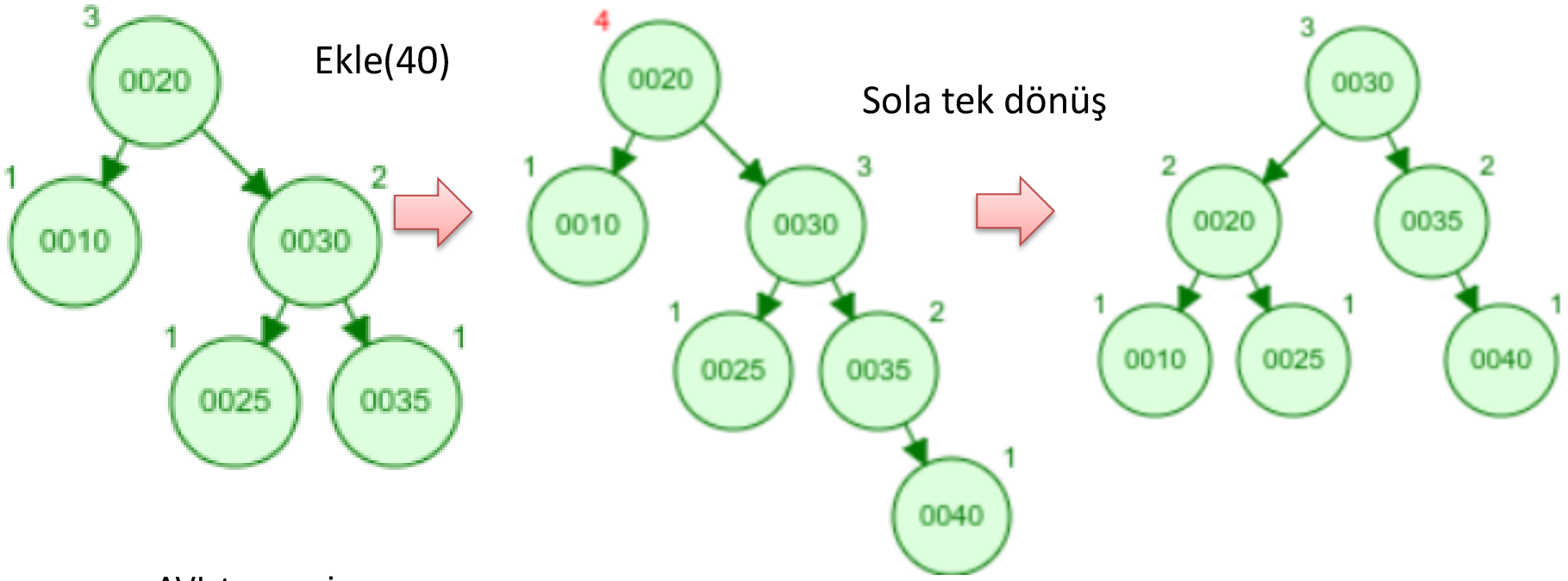




- AVL tree animasyonu:
- https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

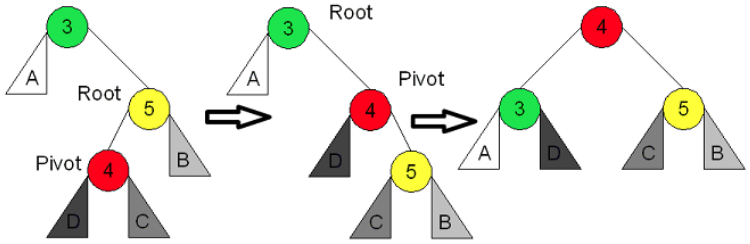
Örnek: sola tek döndürme

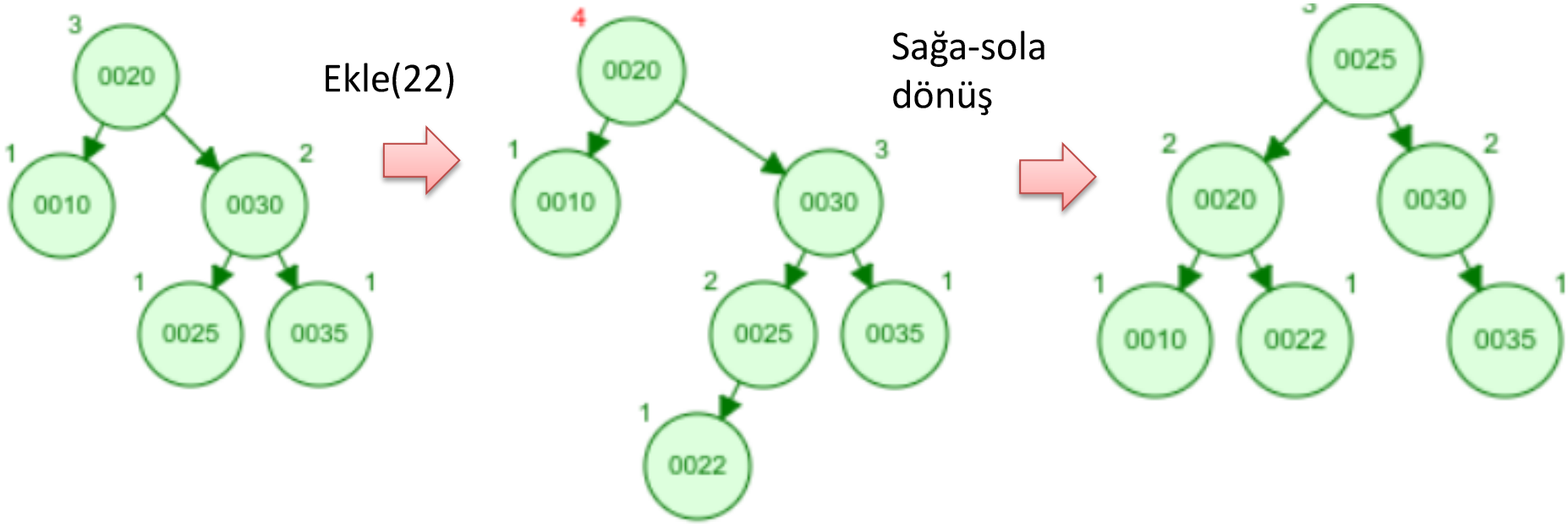




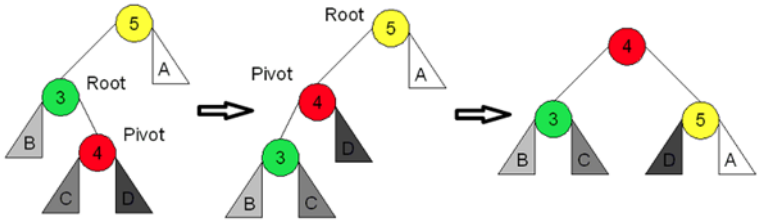
- AVL tree animasyonu:
- https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

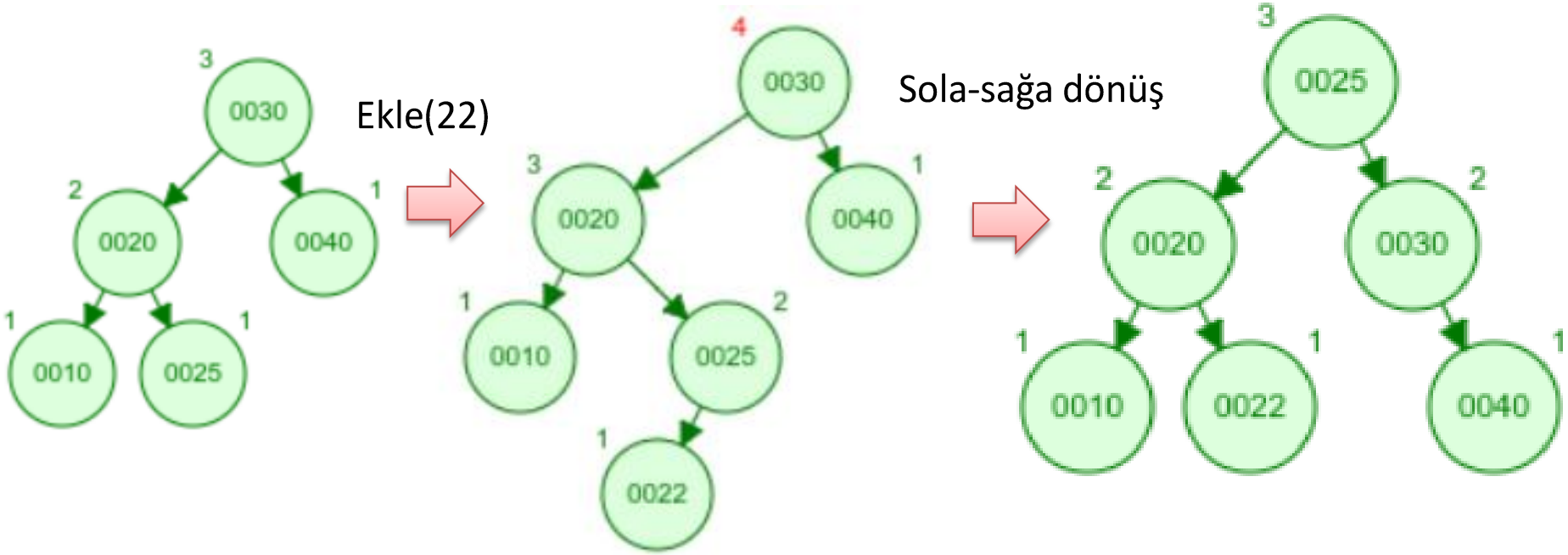
Örnek: sağa-sola döndürme





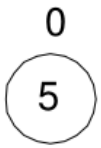
Örnek: sola-sağa döndürme



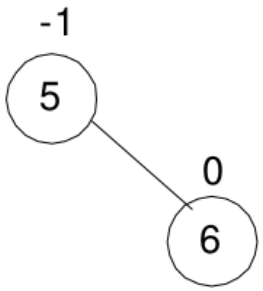


Örnek: Elemanlar: 5, 6, 8, 3, 2, 4, 7

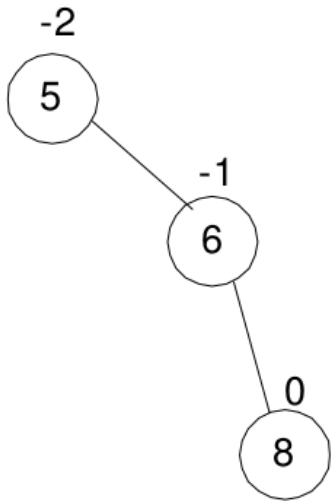
Ekle(5)

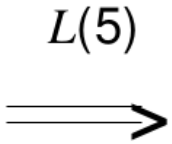


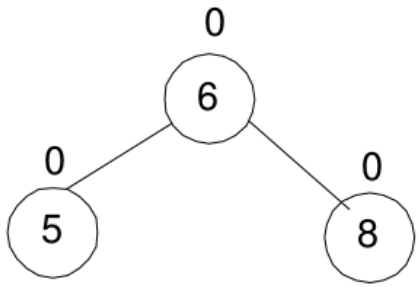
Ekle(6)



Ekle(8)

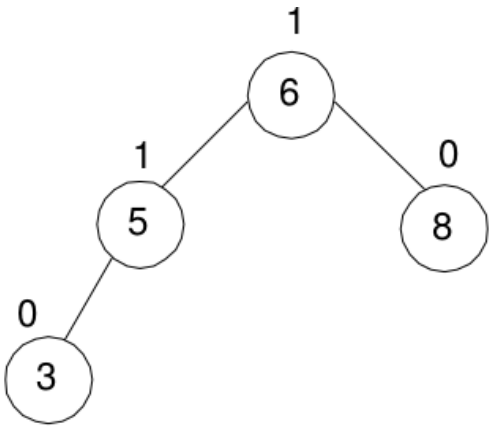




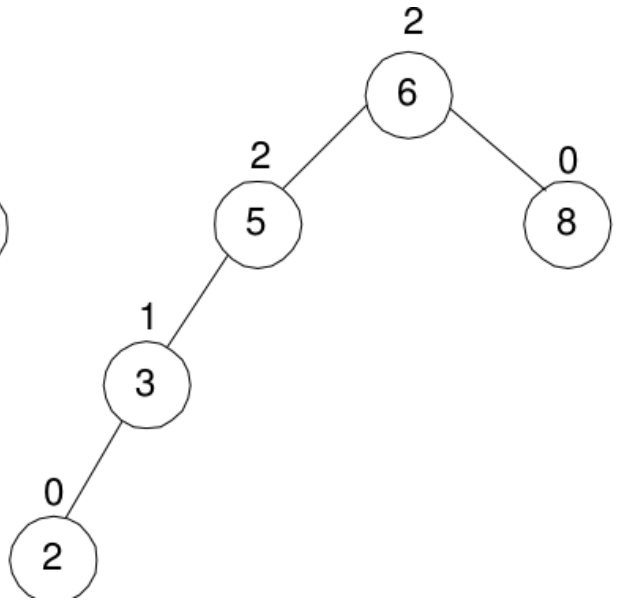


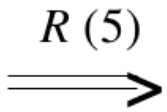
Örnek: Elemanlar: 5, 6, 8, 3, 2, 4, 7

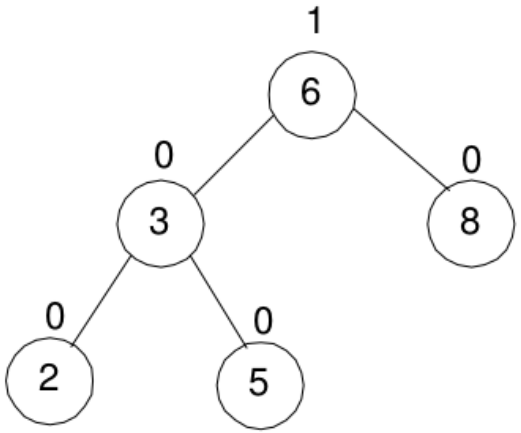
Ekle(3)



Ekle(2)

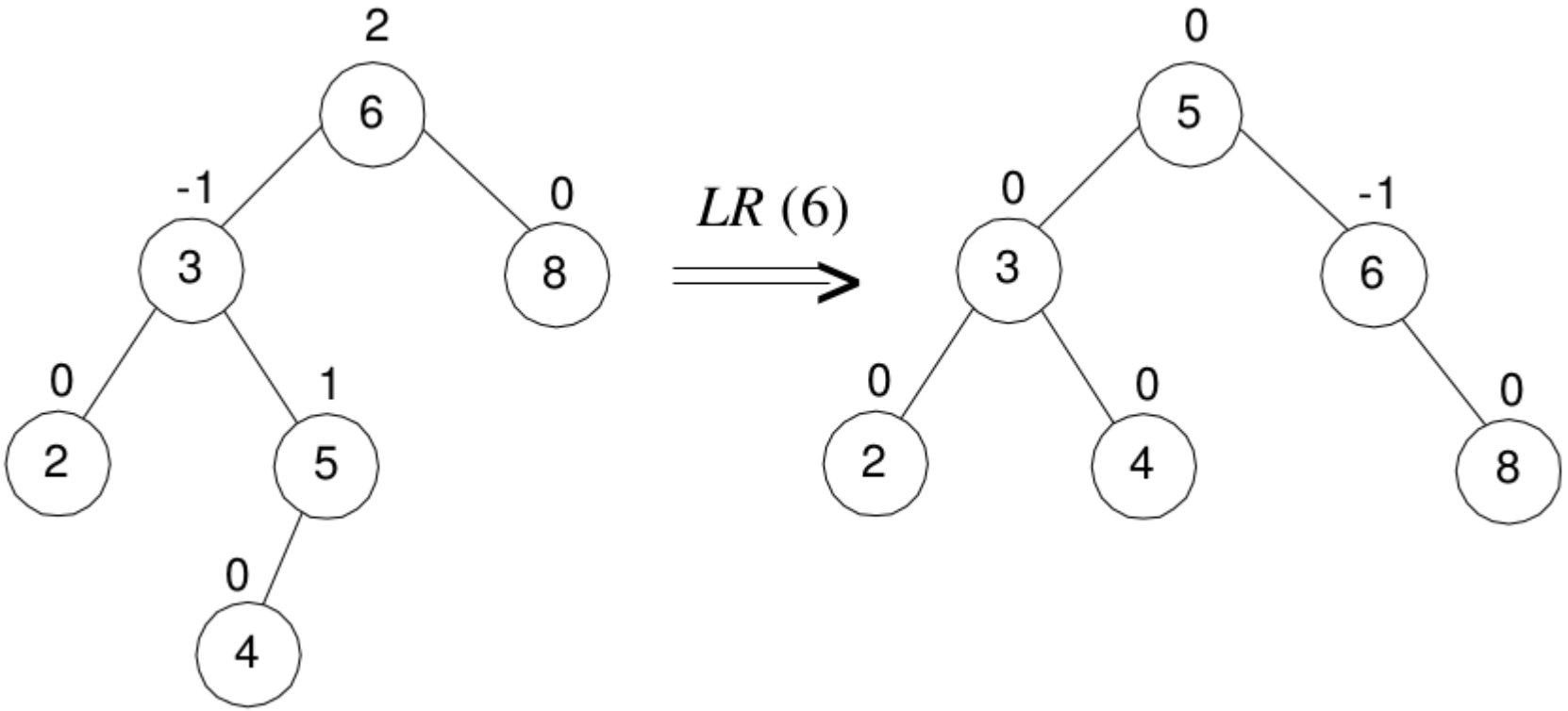






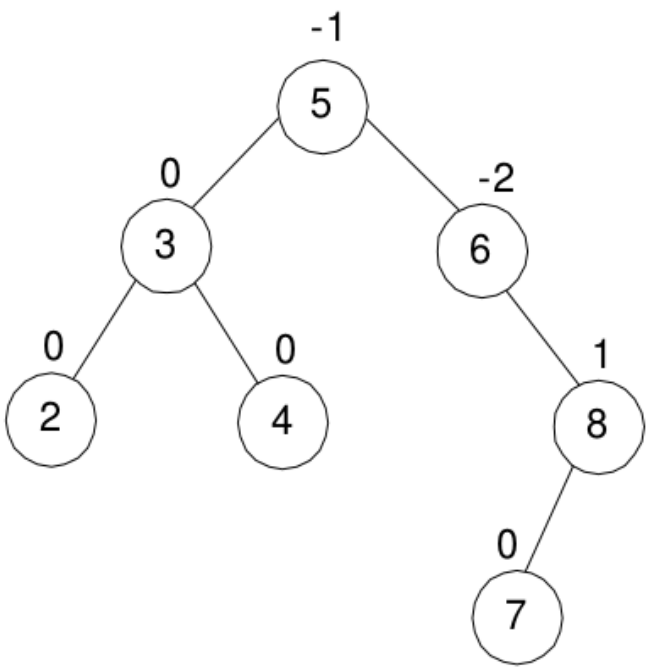
Örnek: Elemanlar: 5, 6, 8, 3, 2, 4, 7

Ekle(4)

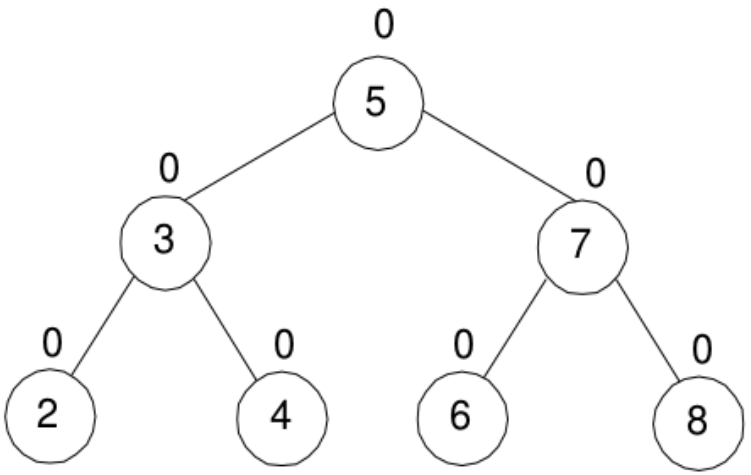


Örnek: Elemanlar: 5, 6, 8, 3, 2, 4, 7

Ekle(7)



$RL\,(6)$ $\Longrightarrow$

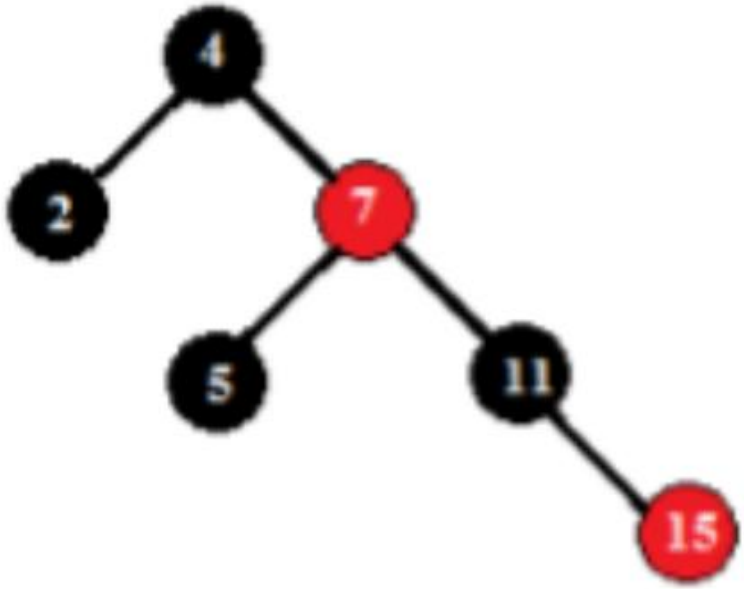


AVL Analizi

- Arama ve Ekleme: O(logn)
- Silme çok karmaşık olmasına rağmen: O(logn)
- Dengeleme işlemi ekleme hızını sadece bir c sabiti kadar yavaşlatır.
- Dezavantajları:
  - Sık yapılan dönüş işlemleri
  - Karmaşıklık

Kırmızı Siyah Ağaçlar (Red-Black Trees)

- Dengelenmiş ağaçların büyük avantajları olduğu gibi özellikle düğüm sildikten sonra dengeyi sağlamak için performans düşürücü işlemleri vardır.
- Bu bölümde herhangi bir güncelleme işinin O(1) zaman aldığı performansı güçlü kırmızı-siyah ağaçlarından bahsedilecektir. Bir kırmızı siyah ağaç aşağıdaki özellikleri içermelidir.
  - Kök her zaman siyahtır.
  - Eğer bir düğüm kırmızı ise çocukları siyah olmalıdır.
  - Yapraklardan köke giden her yolda aynı sayıda siyah düğüm olmalıdır.



Kırmızı Siyah Ağaçlar (Red-Black Trees)

Teorem

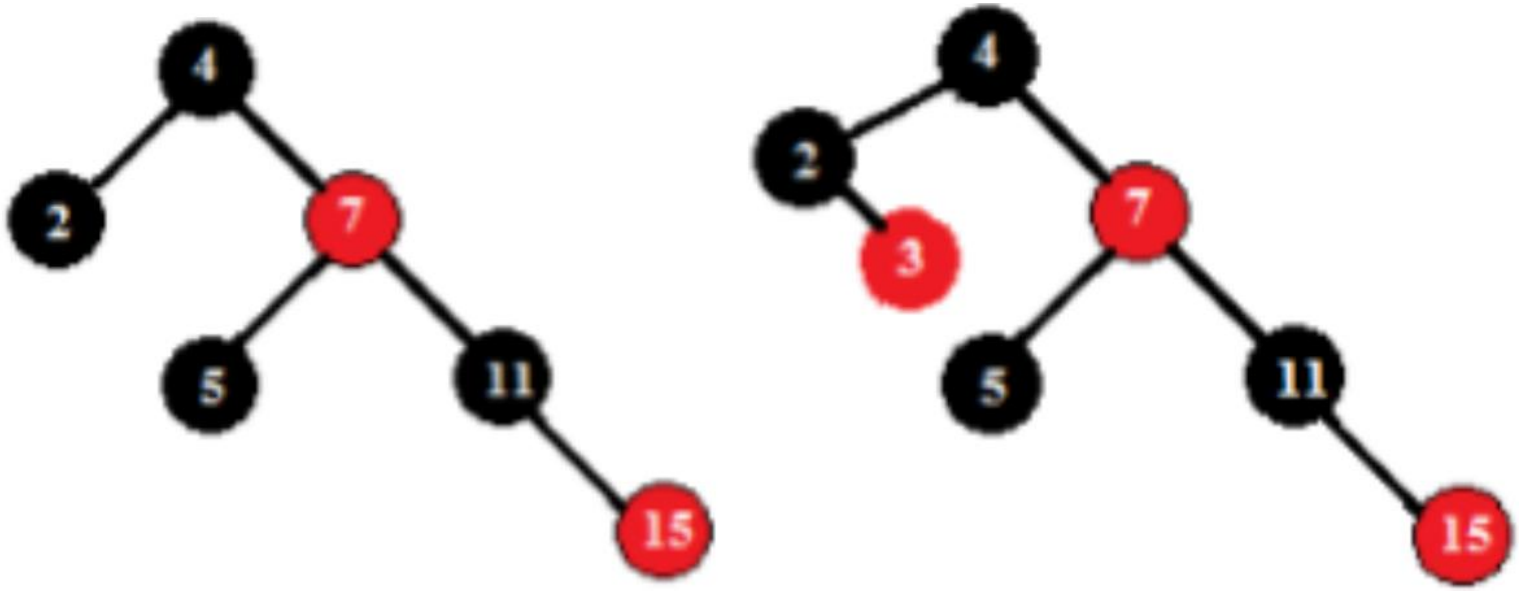
- k adet anahtara sahip bir kırmızı-siyah ağacın yüksekliği
- $h \leq 2\log(k+1)$ ile ifade edilir.

Ekleme İşlemi

- Ekleme işleminde yeni düğüm kırmızı olarak işaretlenir.
- Ekleme işleminde yukarıda belirtilen 3 özellik göz önünde bulundurulmalıdır. Aksi halde düğüm rengi değiştirilmeli ya da döndürme işlemi yapılmalıdır.

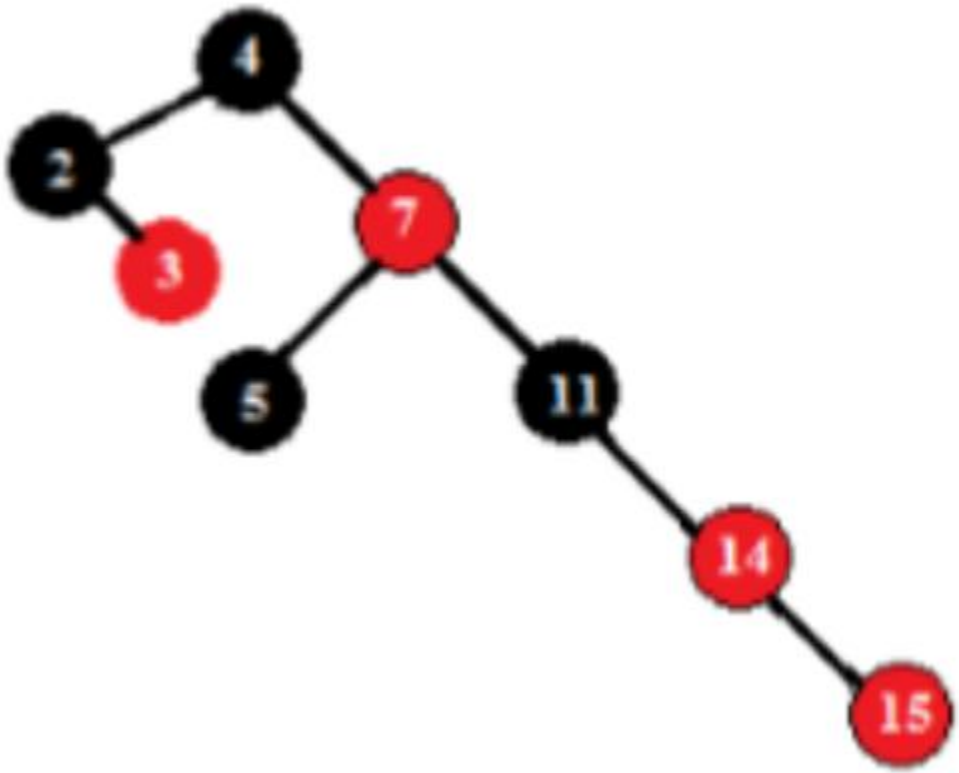
Örnek:

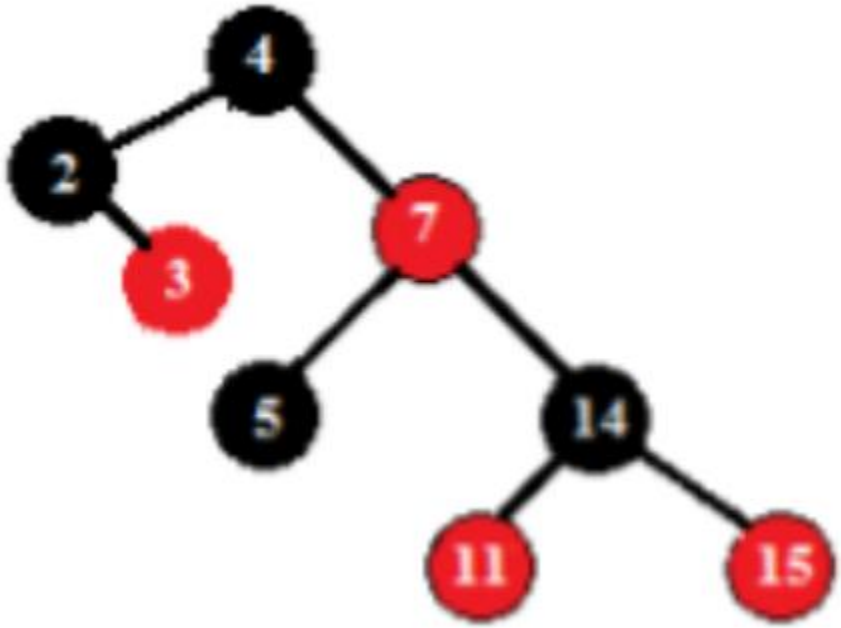
- Örneğin aşağıdaki ağaca 3 sayısı ekleneceği sırada önce kökten başlanır 3<4 olduğu için sola, 3>2 olduğu için sağa ilerlenir ve kırmızıya boyanmış olan 3 düğümü aşağıdaki gibi eklenmiş olur. Ağacın özelliği bozulmadığı için rotasyon işlemi uygulanmaz.



Örnek:

Fakat son ağaca 14 sayısı eklendiğinde “Eğer bir düğüm kırmızı ise çocukları siyah olmalıdır” kuralı bozulmuştur. Ağacı düzeltmek için 14 ile 11 düğümleri yer değiştirerek rotasyon işlemi uugulanacaktır.





Algoritmanın Analiz Edilmesi

- AVL'nin aksine burada
- Çocuk düğümler arasındaki derinlik farkı ≤ 2
- Her durumda bu koşul sağlanmaktadır.
- Herhangi bir düğümü arama işlemi: O(logn)
- Bir düğümün eklenmesi: O(logn)
- Bir düğümün silinmesi: O(logn)
- Kırmızı-Siyah ağaçlar en kötü durumda O(logn) garanti etmektedirler.